

Music Database Schema and Data (MusicDB)

I have developed a complete SQL schema and sample data for a comprehensive music database, created as a final project for a Database Security class. The database is designed to model a complex music ecosystem, capturing intricate relationships between artists, bands, albums, tracks, producers, record labels, and a detailed genre/subgenre taxonomy.

Project Overview

This project demonstrates the design and implementation of a normalized relational database that goes beyond a simple artist-album-track model. It addresses the complexities of the music industry, such as:

- Distinguishing between solo artists and bands.
- Managing many-to-many relationships (e.g., tracks with multiple producers, albums belonging to multiple subgenres).
- Tracking the relationships between genres and a more granular, overlapping subgenre classification.
- Connecting all entities to the record labels that released them.

The database is populated with over 30 sample albums, 30 tracks, 50 producers, and 15+ artists/bands, representing a wide range of genres from jazz and rock to hip-hop and electronic music.

Key Features

- **Comprehensive Schema:** Includes tables for Artist, Band, Album, Track, Producer, Label, Genre, and Subgenre.
- **Many-to-Many Relationships:** Properly implemented junction tables (Artist_Subgenre, Album_Producer, Track_Subgenre, etc.) to handle complex relationships without data redundancy.
- **Normalized Structure:** Designed to minimize data duplication and ensure referential integrity through the use of primary and foreign keys.
- **Real-World Data:** Populated with realistic sample data featuring well-known artists (David Bowie, Kendrick Lamar, The Beatles), albums (*To Pimp a Butterfly*, *Discovery*), producers (Dr. Dre, Max Martin, Rick Rubin), and record labels.
- **Detailed Genre System:** A two-level genre system where Genre (e.g., Hip Hop) is linked to many specific Subgenres (e.g., Trap, Jazz Rap, Conscious Hip Hop), allowing for nuanced musical categorization.

Database Structure

The database is structured to represent the music industry's core entities and their connections. The following is a simplified entity-relationship diagram (ERD) overview:

Core Entities:

- Artist / Band: Solo musicians and musical groups.
- Album: A collection of tracks, linked to either an Artist or a Band.
- Track: An individual song, linked to an Album.
- Label: The record company that released an album or track.
- Producer: An individual or group responsible for producing a track or album.
- Genre / Subgenre: A hierarchical classification system for musical styles.

Key Relationships:

- An Artist or Band can have many Albums, but an Album belongs to only one.
- An Album can be produced by many Producers, and a Producer can work on many Albums (handled by Album_Producer).
- A Track can be associated with multiple Producers (handled by Track_Producer).
- Artist, Band, Album, and Track can all be linked to multiple Subgenres, providing flexible and precise categorization.

Getting Started

Prerequisites

- A SQL database management system (DBMS) such as **MySQL** or **MariaDB**.
- A SQL client (like MySQL Workbench, phpMyAdmin, or the command-line client) to execute the scripts.

Setup Instructions

To build and explore this database on your local machine, follow these steps:

1. **Clone the repository** (or download the SQL files).

```
git clone <your-repository-url>
cd music-db-portfolio
```

2. **Run the SQL scripts in order.** The scripts are organized to create the database, define the tables, establish foreign keys, and then insert the data. It is important to execute them in the following sequence to avoid foreign key constraint errors:

- **Create and Link Tables:** Execute `music-db-alter_table+foreign_keys.sql` first. This script creates all the tables and sets up the primary and foreign key relationships.
 - **Insert Core Data:** Populate the tables with data. The order is crucial as some tables depend on others.
 1. `music-db label.sql`
 2. `music-db genre and subgenre.sql`
 3. `music-db artists.sql`
 4. `music-db bands.sql`
 5. `music-db producer.sql`
 6. `music-db albums.sql`
 7. `music-db tracks.sql`
 - **Link Relationships:** Finally, populate the many-to-many junction tables.
 1. `music-db subgenre and producer links.sql`
 2. `music-db connecting fk.sql` (This file also contains some duplicate data but serves as a final step to ensure all link tables are populated.)
3. **Verify the Installation:** After running all scripts, you can use the `music-db show tables.sql` script, which contains a series of SELECT queries to view the final, linked data in a human-readable format. This is a great way to check if everything loaded correctly.

Sample Data Highlights

The database includes a curated set of data covering over 70 years of music history:

- **Artists:** David Bowie, Kendrick Lamar, Aphex Twin, Eminem, Björk, Miles Davis.
- **Bands:** The Beatles, Radiohead, Daft Punk, Kraftwerk, Outkast, Nine Inch Nails.
- **Albums:** *Blackstar*, *To Pimp a Butterfly*, *Sgt. Pepper's Lonely Hearts Club Band*, *Discovery*, *Madvillainy*.
- **Producers:** Dr. Dre, George Martin, Rick Rubin, Max Martin, Madlib.
- **Labels:** Columbia, Interscope, Warp, Def Jam, Sub Pop.

This data demonstrates the database's ability to handle complex scenarios, such as a track like "Alright" by Kendrick Lamar being produced by six different producers (Dr. Dre, Sounwave, Thundercat, Flying Lotus, Pharrell Williams, and Terrace Martin).

Acknowledgments

This database was created as a portfolio project for a Database Security course. The schema and data were designed to demonstrate a deep understanding of relational database design, normalization, and complex data modeling.

To access this database, it is available publicly at the following link:

<https://github.com/x-alted/music-sample-database>